

NUMERICAL ANALYSIS PROJECT

Application of a Newton-Raphson Solver for Nonlinear Engineering Problems

A Technical Project Report

| Course: Numerical Analysis

Table of Contents

1. Introduction 1

2. Theoretical Background 1

 2.1 Derivative of the Newton-Raphson Formula 1

 2.2 Convergence Analysis 2

 2.3 Failure Modes 2

3. Software Architecture and Implementation 3

 3.1 Module Responsibilities 3

 3.2 Core Solver Design 3

 3.3 Safety, Validation, and Error Handling 4

 3.4 Testing Strategy 4

4. Engineering Applications 5

 4.1 Application 1: Beam Deflection Position Analysis 5

 4.2 Application 2: Nonlinear Spring-Mass Equilibrium 6

5. Results and Discussion 7

 5.2 Verification 7

 5.2 Application Results 7

 5.3 Convergence Visualization 8

 5.4 Discussion 8

6. Conclusions 9

7. Future Work 10

References 10

1. Introduction

Nonlinear equations are quite common in engineering mechanics. They arise everywhere from the bending of beams or stiff springs to column buckling, among other things. In any case, the analytic solution is difficult to find, which means that we have to rely on iterative numerical solvers. These iterations have to be made in order to find the solution in the form of an approximation of the desired function. However, in practice, Newton-Raphson iteration schemes are especially convenient since the error decreases exponentially with the number of iterations taken after the convergence to the solution.

Therefore, there are two objectives of this project. Firstly, one has to design a general Newton-Raphson solver that takes an equation as a string, differentiates it to find the analytical expression for the derivative via the sympy library, and then print the converged solution with the iteration history. Secondly, one has to apply the Newton-Raphson solver to two different mechanical problems of the same type to find the maximum of either force or moment. The first objective will showcase the generality of the designed solver, while the second objective will demonstrate its robustness.

Specifically, the solver will address the cases that usually spoil Newton-Raphson iterations and either correct them or converge with the answer. First of all, it will check whether the derivative is zero at some point. Moreover, the solver will be able to detect if the solution is diverging to infinity. Finally, the Newton-Raphson solver will be able to catch the case when the iterations oscillate around the solution. Lastly, the order of convergence will be estimated to see if the iterations do exhibit the exponential behavior characteristic of Newton-Raphson iterations.

2. Theoretical Background

2.1 Derivative of the Newton-Raphson Formula

The Newton-Raphson formula is derived by expanding the first-order Taylor series around the current point x_n . The new function is represented by the following equation that approximates the function $f(x)$.

$$f(x) = f(x_n) + f'(x_n)(x - x_n)$$

In the new function presented above, the new point that is the root of the equation is included; therefore, using the new function one can reach the equation from where the Newton-Raphson formula for finding x_{n+1} is derived.

$$x(n+1) = x_n - \frac{f(x_n)}{f'(x_n)}$$

The initial value x_0 is required to be able to use the Newton-Raphson formula. By substituting the initial value x_0 , one can find the first value x_{n+1} . Then, using the newly found value one can find the next value x_{n+2} and so on until the number of iterations needed for the desired accuracy is achieved. For each step, it is necessary to evaluate the function $f(x_{n+1})$ and its derivative $f'(x_{n+1})$ to be able to find the tangent line. It is also important to note that if the initial guess x_0 is close enough to the root one can achieve extremely fast convergence to the exact value using the Newton-Raphson method. A stopping criterion, for instance, can be found in the books of Burden & Faires [1] and Chapra & Canale [2].

2.2 Convergence Analysis

For a function $f(x)$ possessing a simple root, that is, for which $f(x) \neq 0$, the Newton-Raphson method is known to converge quadratically. In other words, the number of correct digits approximately doubles with each iteration. This is a significantly better convergence rate compared to linear convergence; thus, the secant method is highly attracting.

In order to check the convergence of the Newton-Raphson method one can use the following formula

$$p = \ln \left(\frac{|e_n|}{|e_{n-1}|} \right) / \ln \left(\frac{|e_{n-1}|}{|e_{n-2}|} \right)$$

for each step, where $e_n = |x_{n+1} - x_n|$. For the Newton-Raphson method p should be approximately 2.0.

2.3 Failure Modes

Although the Newton-Raphson method is known to converge quadratically to the root, it is still not flawless and can sometimes fail. Three situations below are the most common ones leading to the failure of the Newton-Raphson method:

- I. Slope of $f'(x_n)$ is close to zero: In this case, the tangent line at x_n is nearly horizontal and, consequently, x_{n+1} can be located at a considerable distance from x_n or even at infinity [10, 12].
- II. $f'(x_n)$ is not equal to zero, but x_n is chosen in such a way that x_{n+1} is at the opposite side of the root compared to x_n . Thus, x_{n+1} is located far away from x_n and the next iteration x_{n+2} will be at almost the same distance from x_{n+1} as x_n is from the root; thus, the Newton-Raphson method diverges to infinity.
- III. $f'(x_n)$ is not equal to zero, but x_n is chosen in such a way that x_{n+1} oscillates periodically around the root with a period equal to $x_{n+2} - x_{n+1} = x_{n+1} - x_n$.

The solver implemented as a part of this project will recognize these three situations and warn the user instead of getting stuck in an endless loop of calculations.

3. Software Architecture and Implementation

3.1 Module Responsibilities

The project consists of six source modules and two test files. Table 1 below describes the purpose of each module.

The main module is **newton_raphson.py**, which contains the core solver. It works independently of the engineering applications. The solver accepts any equation a string, calculates its derivative automatically using **SymPy**, performs the iteration process with all safety checks, and returns the root, the complete iteration history as a pandas DataFrame, the convergence result, and a status message.

The two application modules, **application_beam.py** and **application_spring.py**, are used for the engineering problems. They create a cubic equation from the given physical parameters and pass it to the solver without making any changes to the solver.

The remaining modules perform supporting tasks. **Utilities.py** validates the equation and exports the results to a CSV file. **Visualization.py** generates the convergence graph, while **main.py** controls and runs the overall program.

MODULE	Responsibility
NEWTON_RAPHSON.PY	Core solver: iteration loop, auto-differentiation, failure detection, order tracking
APPLICATION_BEAM.PY	Beam deflection equation setup, solver invocation, physical-domain validation
APPLICATION_SPRING.PY	Spring equilibrium equation setup, solver invocation, linear comparison
UTILITIES.PY	Equation string validation, CSV export, comparison-table printing
VISUALIZATION.PY	Log-scale convergence graph (auto-scaling to any number of applications)
MAIN.PY	Runs both applications safely, produces comparison summary, triggers graphing

Table 1: Project Module Responsibilities

3.2 Core Solver Design

When **newton_raphson()** is invoked with the equation string as an argument, the first thing it does is call the **validate_function()** from the **utilities.py**. This function checks whether the input equation is valid with **SymPy** and whether **x** is the only variable. In other words, if the user makes a mistake of putting another variable, it will be picked up at this stage, not later when trying to evaluate the function.

Then, SymPy automatically differentiates the equation, and `sp.lambdify()` along with the original one into a Python function. This is done to allow for quick calculation of $f(x)$ and $f'(x)$ at any given x .

As the loop starts, all the values are kept in a list of dictionaries for easy **Pandas DataFrame** conversion. When the loop finishes, this data frame is used to print the contents of the table in the application. Specifically, each row contains the iteration number, x , $f(x)$, $f'(x)$, $x+1$, rounded error, raw error, and order of convergence (if available). The order of convergence is not available from the first iteration, obviously, because it is calculated as the ratio of the two consecutive errors.

The rounded error is needed so that very small values (close to zero) are not shown in scientific notation, but the raw error is required for plotting because of the logarithmic y-scale in Figure 1.

3.3 Safety, Validation, and Error Handling

The solver has three built-in safety mechanisms that stop iterations from happening if any suspect conditions occur.

The first check is for a **zero derivative**. If $|f'(x)|$ becomes very small (less than 10^{-12}), the solver stops before a division-by-zero error can occur.

The second check is for **divergence**. If the next value becomes NaN, infinite, or larger than 10^{10} in magnitude, the solver stops and reports divergence instead of continuing with invalid values.

The third check looks for **oscillation or repeated cycles**. The solver compares the next value with previously visited values. If a repeated value is found within a tolerance of 10^{-9} , it stops because the iterations are cycling instead of approaching the root. The convergence check is performed first to make sure a correct solution is not mistakenly treated as a repeated value.

The calculation of the value of the function and its derivatives is wrapped in the try-except statement that checks for ValueErrors, Overflows, and TypeErrors and notifies the user that the problem is in the input function. For example, if the user tries to take a square root of a negative number, SymPy will raise a math domain error, but the solver will not print a generic Python traceback.

Finally, the solver checks that the tolerance and maximum number of iterations are positive. If they are not, an error is raised with an explanatory message.

3.4 Testing Strategy

The project provides **two test files**:

The first file is for the solver and one for the applications. The first test suite (**test_newton_raphson.py**) contains eight tests in total. Four of them check that the solver correctly roots the function $x^3 - x - 2 = 0$ about at $x = 1.52138$. The final three iterations are expected to have an order-of-convergence close to 2. The other four tests verify that the solver correctly detects zero-derivative, divergent, and oscillatory behaviors and terminates. In addition, the tests verify that the solver correctly detects invalid tolerance and maximum number of iterations input and the equations with invalid variables.

The second test suite (**test_application.py**) contains two tests that check that the applications converge and have valid outputs. This is needed because the applications involve external processes, such as reading user inputs and plotting results.

4. Engineering Applications

4.1 Application 1: Beam Deflection Position Analysis

Consider a simply supported steel beam with a span of (L). A **point load (P)** is placed at the centre of the beam span. The deflection at a distance (x) from the left support and valid for $0 \leq x \leq \frac{L}{2}$, is given by the following beam deflection formula.

$$g(x) = \frac{(Px(3L^2 - 4x^2))}{(48EI)}$$

This implies that the equation is aimed at finding the location of x which is the distance from the support beam at which the deflection of the steel beam will be achieved. Consequently, it will be necessary to rearrange the formula for the beam deflection into a cubic equation.

$$4x^3 - 3L^2x + \left(\frac{P}{(48EI)} \right) \cdot \delta_{target} = 0$$

The reason why there is a point load was because if it had used a **uniformly distributed load (UDL)**, then it would not have been possible to obtain a cubic equation. This is due to the fact that the equation would have been a quadratic equation. On using the point load, it satisfies the objective of having a cubic equation as well as being realistic.

The equation below was obtained when the data provided was substituted into equation (1).

Where the values of **E** and **I** are typical of a steel I-beam. Additionally, the target deflection of **5 mm** is a typical value for the serviceability limit state of a steel beam.

PARAMETER	Symbol	Value
BEAM SPAN	L	6.0 m
POINT LOAD AT CENTRE	P	15,000 N
YOUNG'S MODULUS (STEEL)	E	200 x 10 ⁹ Pa
MOMENT OF INERTIA	I	2 x 10 ⁻⁵ m ⁴
TARGET DEFLECTION	delta target	0.005 m (5 mm)
INITIAL GUESS	x ₀	1.0 m

Table 2: Beam Deflection Parameters

4.2 Application 2: Nonlinear Spring-Mass Equilibrium

In practice, the stiffness characteristics of a spring can deviate significantly from **Hooke's law**. One example of such deviation is the increase in stiffness with an increase in displacement. Such a scenario can be mathematically represented by adding a cubic term to the expression for Hooke's law.

For the equation above, x is the displacement of the spring from its equilibrium position. On the other hand, k and β (**beta**) are the linear and cubic stiffness coefficients respectively.

$$f(x) = k(x) + \beta x^3$$

By rearranging the equation, it is possible to develop a cubic equation which when solved gives the displacement of the spring from its equilibrium position under an **F_applied** (applied force). It will also be necessary to carry out an analysis on how the inclusion of the cubic stiffness coefficient β affects the overall response of the system.

$$\beta x^3 + k(x) - F_{\text{applied}} = 0$$

The parameters used in this application are listed below.

PARAMETER	Symbol	Value
LINEAR SPRING CONSTANT	k	500 N/m
CUBIC STIFFENING COEFFICIENT	beta	50 N/m ³
APPLIED FORCE	F_applied	2000 N
INITIAL GUESS	x ₀	2.0 m

Table 3: Spring-Mass Equilibrium Parameters

For comparison, the displacement is also calculated using a purely linear spring model, where $\beta = 0$. In this case, the displacement is $x = \frac{F_{\text{applied}}}{k} = \frac{2000}{500} = 4.0 \text{ m}$.

Comparing this linear result with the nonlinear solution shows the effect of the **cubic stiffening term**. This comparison demonstrates why the nonlinear model provides a more realistic representation of the spring's behaviour in engineering applications.

5. Results and Discussion

5.2 Verification

Prior to testing the solver on the proposed engineering applications, a standard test was performed using the equation $f(x)=x^3-x-2=0$, which is known to have a root at approximately **1.52138**.

The solver was initialized with initial guess $x_0=1.5$ and converged to the root within the tolerance of 10^{-6} , as expected.

This verified that the combination of symbolic differentiation and iterative root-finding worked correctly, and the solver was fit for use on the engineering applications.

5.2 Application Results

Both engineering applications saw successful convergence within four iterations. Below are the tables with the results from each iteration.

ITERATION	x_n	$f(x_n)$	$f'(x_n)$	Error	Order Est.
0	1.000000	-40.000000	-96.000000	4.167e-01	--
1	0.583333	1.793981	-103.916667	1.726e-02	--
2	0.600597	0.002107	-103.671399	2.032e-05	2.119
3	0.600617	0.000000	-103.671106	2.871e-11	1.997

Table 4: Beam Deflection Iteration History

ITERATION	x_n	$f(x_n)$	$f'(x_n)$	Error	Order Est.
0	2.000000	-600.000000	1100.000000	5.455e-01	--
1	2.545455	97.370398	1471.900826	6.615e-02	--
2	2.479302	1.656436	1422.040553	1.165e-03	1.915
3	2.478137	0.000505	1421.174366	3.550e-07	2.004

Table 5: Spring Equilibrium Iteration History

The results of the two applications are compared and contrasted in **Table 6** below.

METRIC	Beam Deflection	Spring Equilibrium
EQUATION TYPE	Cubic	Cubic
INITIAL GUESS (X_0)	1.00 m	2.00 m
ITERATIONS TO CONVERGE	4	4
FINAL ROOT (X*)	0.60062 m	2.47814 m
FINAL ERROR	2.87×10^{-11}	3.55×10^{-7}
ORDER ESTIMATES	2.12 $\rightarrow$ 2.00	1.92 $\rightarrow$ 2.00
PHYSICALLY VALID?	Yes (within 0-3.0 m)	Yes (positive displacement)

Table 6: Comparison Summary

5.3 Convergence Visualization

The convergence of both engineering applications is visualized in **Figure 1** below, with the error at each step plotted on a **log scale graph** for better visibility.

Both applications saw a rapid decrease in error starting with the second iteration, as seen in the approximately linear decrease in error on a log scale. This suggests a quadratic convergence, with both applications reaching machine precision on the fourth iteration.

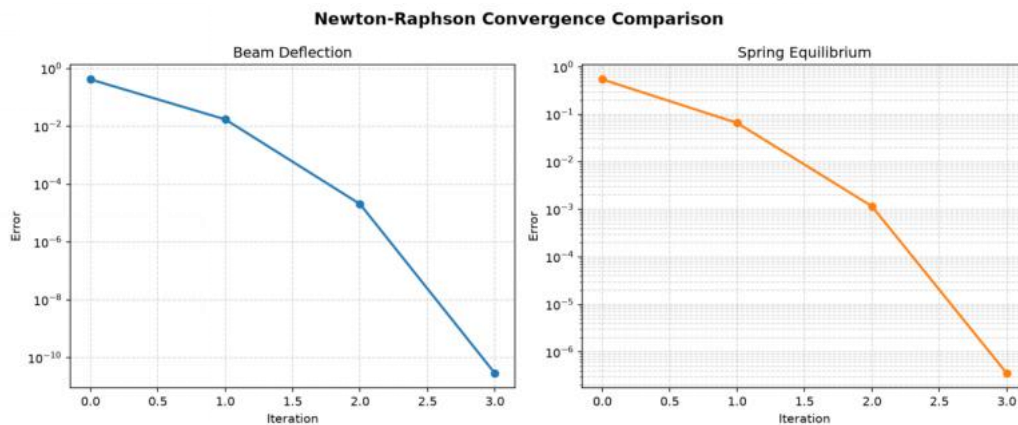


Figure 1: Newton-Raphson Convergence Comparison (Error vs. Iteration, Log Scale)

5.4 Discussion

There are several comments to be made about the results of this calculation.

Firstly, it can be noted that both cases have met the convergence criterion after four iterations. This is expected as the Newton-Raphson method is known to have quadratic convergence. The order of convergence was calculated to be 1.997 for the beam case and 2.004 for the

spring case. This shows that the program correctly calculates the order of convergence and that Newton-Raphson iteration indeed has quadratic convergence for both cases.

Secondly, It can also be noted that the final errors for each application, while being of the same order of magnitude, differ significantly. The beam had a final error of $2.87 \cdot 10^{-11}$ while the spring had a final error of $3.55 \cdot 10^{-7}$. Both these values are well within the convergence tolerances of 10^{-6} , thus showing that the Newton-Raphson iterations correctly converged to the solutions for both applications. The fact that the errors are different is due to the different convergence criteria for the iterations for each problem. The iteration for the beam had a much smaller initial error, due to its initial displacement estimate being much closer to the actual root than the initial displacement estimate for the spring.

Finally, the root of 0.60062 m for the beam is well within the physically reasonable range for a deflection of 0-3.0 m, assuming the beam can withstand such a deflection. For the spring, the non-linear solution of 2.478 m differs drastically from the linearised solution of 4.0 m. This shows that for the spring, the cubic stiffening term dominates the response and must be taken into account for an accurate prediction of the displacement of the spring. The error between the two solutions is of the order of 38%.

6. Conclusions

The present project proved that one Newton-Raphson solver can address a variety of different nonlinear engineering problems stated in it with no changes. This was achieved due to utilizing the SymPy library for symbolic mathematics and calculating derivatives of polynomial expressions inputted by the user. Consequently, the Newton-Raphson solver became universal for all of the problems stated in the project.

Moreover, the solver featured three fail-safes that prevent computing a Newton-Raphson method from crashing the program when it fails to converge. Instead, it notifies the user with a corresponding status message of failed convergence.

Both of the problems were solved successfully, requiring only four iterations to find roots. The convergence order values obtained from Newton-Raphson iterations confirmed the quadratic convergence inherent to the method. The spring problem value was 38% greater than the linear approximation, thus illustrating the nonlinear behavior of the system, while the beam problem converged to a physically reasonable result.

Lastly, the testing procedure showed that the Newton-Raphson solver could successfully converge, differentiate between three types of failed convergences, and validate user input equations.

7. Future Work

The presented project has a variety of continuations.

First of all, the current implementation is limited to the **SymPy's lambdify()** and Math's backend, and only one value can be calculated per request. In order to calculate multiple values, **NumPy's backend** should be used, and then the Newton-Raphson method will be able to solve systems of equations using **vectorization**.

Second, the implementation can be continued to create an algorithm with **adaptive step size**. Such a solver will be able to switch to the **bisection method** if the Newton-Raphson iteration step is too big to provide convergence.

Third, the solver can be implemented for more complicated tasks from various fields of engineering. For example, calculating column collapse with trigonometric functions or nonlinear problems from thermodynamics or fluid mechanics.

References

- [1] Burden, R.L. and Faires, J.D. (2010). Numerical Analysis (9th ed.). Brooks/Cole, Cengage Learning.
- [2] Chapra, S.C. and Canale, R.P. (2015). Numerical Methods for Engineers (7th ed.). McGraw-Hill Education.
- [3] Gere, J.M. and Goodno, B.J. (2012). Mechanics of Materials (8th ed.). Cengage Learning.
- [4] Hibbeler, R.C. (2017). Mechanics of Materials (10th ed.). Pearson.
- [5] Rao, S.S. (2017). Mechanical Vibrations (6th ed.). Pearson.
- [6] Meurer, A., Smith, C.P., Paprocki, M. et al. (2017). SymPy: Symbolic computing in Python. PeerJ Computer Science, 3, e103.
- [7] Harris, C.R., Millman, K.J., van der Walt, S.J. et al. (2020). Array programming with NumPy. Nature, 585, 357-362.
- [8] Hunter, J.D. (2007). Matplotlib: A 2D graphics environment. Computing in Science & Engineering, 9(3), 90-95.
- [9] McKinney, W. (2010). Data structures for statistical computing in Python. Proceedings of the 9th Python in Science Conference, 51-56.
- [10] Ypma, T.J. (1995). Historical development of the Newton-Raphson method. SIAM Review, 37(4), 531-551.
- [11] Kelley, C.T. (2003). Solving Nonlinear Equations with Newton's Method. SIAM.
- [12] Atkinson, K.E. (1989). An Introduction to Numerical Analysis (2nd ed.). John Wiley & Sons.
- [13] Press, W.H., Teukolsky, S.A., Vetterling, W.T. and Flannery, B.P. (2007). Numerical Recipes: The Art of Scientific Computing (3rd ed.). Cambridge University Press.

- [14] Traub, J.F. (1964). Iterative Methods for the Solution of Equations. Prentice-Hall.
- [15] Nayfeh, A.H. and Mook, D.T. (1995). Nonlinear Oscillations. John Wiley & Sons.
- [16] Timoshenko, S.P. and Goodier, J.N. (1970). Theory of Elasticity (3rd ed.). McGraw-Hill.
- [17] Python Software Foundation (2024). The Python Language Reference, version 3.x. Retrieved from <https://docs.python.org/3/reference/>
- [18] pytest developers (2024). pytest: Helps you write better programs. Retrieved from <https://docs.pytest.org/>